Convolutional Neural Networks (CNN)

Introduction to Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a specialized class of deep learning algorithms that have achieved state-of-the-art results across a wide variety of problem domains. While they are most famous for their performance in computer vision, their applications are vast and limitless.

Key Applications

1. **Computer Vision & Image Classification**
    - **Core Function:** The primary use of CNNs is taking an image as input and assigning it a label that summarizes its content.
    - **Performance:** In image classification tasks, CNNs almost always outperform standard neural networks.
2. **Gaming and AI Agents**
    - CNNs allow AI agents to "see" game screens and learn strategies without prior knowledge of the game rules.
    - **Examples:**
        - Playing arcade games like **Atari Breakout**.
        - Guessing drawings in **Pictionary** (e.g., Google's Quickdraw).
        - Mastering complex board games like **Go** (DeepMind).
3. **Voice User Interfaces**
    - Used in models like **WaveNet** to generate realistic, human-like audio from text.
    - Can be trained to mimic specific human voices.
4. **Natural Language Processing (NLP)**
    - While Recurrent Neural Networks (RNNs) are more common here, CNNs are also used to extract information from text.
    - **Sentiment Analysis:** determining if a writer is happy/sad or likes/dislikes a movie.
5. **Real-World Automation**
    - **Drones:** Navigating unfamiliar territory and analyzing streaming video (e.g., delivering medical supplies).
    - **Self-Driving Cars:** Reading road signs and detecting house numbers for mapping.
    - **Digitization:** Decoding images of text, handwritten notes, or historical books.

Summary

CNNs are robust algorithms capable of extracting critical information from raw data (like pixels or audio waves) to solve complex problems, often exceeding human performance in recognition and strategy tasks.

Image Classification Pipeline Review

1. **Data Preparation**
    - Load and visualize the dataset.
    - Pre-process data by normalizing and converting it into tensors for the network.
2. **Research & Design**
    - Investigate if similar tasks have been solved before.
    - Review existing deep learning models and architectures.
    - Define your model architecture based on this research.
3. **Training Configuration**
    - Select appropriate loss functions and optimizers.
4. **Training & Validation**
    - Train the model using a validation dataset.
    - Save the best-performing model during training.
5. **Testing**
    - Evaluate the model on previously unseen data.
6. **Next Steps**
    - Reflect on learned concepts.
    - Introduction to **Convolutional Neural Networks (CNNs)** for building superior classification models.

MLPs vs. CNNs for Image Classification

- **What is an MLP?**
    - **MLP (Multi-Layer Perceptron)** is a fundamental type of neural network consisting of fully connected layers (often referred to as a "Fully Connected Network").
    - It requires input data to be flattened into a single-dimensional list of numbers, treating the input as a simple vector.
- **Performance Context**
    - MLPs perform well on the MNIST dataset because these images are "clean": digits are centered, uniform in size, and pre-processed.
    - However, MLPs struggle significantly with real-world, "messy" data where objects can appear anywhere in the image or vary in size.
- **The Critical Limitation of MLPs**
    - To process an image, an MLP must destroy its structure by converting the 2D grid into a 1D vector.
    - This loss of **spatial structure** means the network has no knowledge of pixel proximity (i.e., it doesn't know which pixels are next to each other), making it difficult to detect patterns in complex images.
- **The Power of CNNs (Convolutional Neural Networks)**
    - **Consistently outperform MLPs** on general image classification tasks.
    - Designed specifically for multi-dimensional data, allowing them to take raw images as input without flattening them first.
    - They understand **spatial relationships**, recognizing that pixels close to each other are heavily related.

Technical Report: Transitioning from Multi-Layer Perceptrons to Convolutional Neural Networks

1. Limitations of Conventional Multi-Layer Perceptrons (MLPs)

In previous analyses using the MNIST dataset, Multi-Layer Perceptrons (MLPs) demonstrated high accuracy (less than 2% error) in classifying handwritten digits. However, applying MLPs to more complex, real-world image classification tasks reveals significant scalability and structural limitations.

Two primary issues arise when scaling MLPs for sophisticated image analysis:

1. **Computational Complexity:** MLPs require a vast number of parameters. For instance, a simple network for small 28x28 pixel images already necessitates over 500,000 parameters. As input resolution increases,

the computational cost grows exponentially.

2. **Loss of Spatial Information:** To feed an image into an MLP, the 2D matrix must be flattened into a 1D vector. This process destroys critical spatial relationships—knowledge of how pixels relate to their neighbors—which is essential for recognizing patterns and structures.

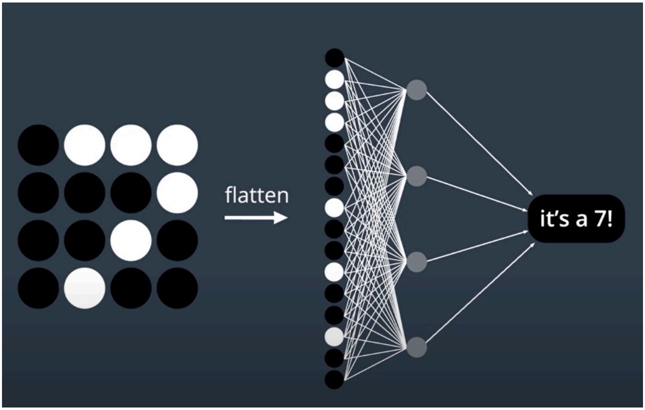


Figure 1: Traditional MLP architecture where 2D spatial information is lost during the flattening process.

2. Structural Improvements: Sparsity and Local Connectivity

To address these limitations, we explore a new architectural paradigm that preserves the 2D structure of the input data: the Convolutional Neural Network (CNN).

Consider a simplified example using a 4x4 image grid. In a fully connected dense layer, every hidden node processes the entire image simultaneously, leading to significant redundancy.

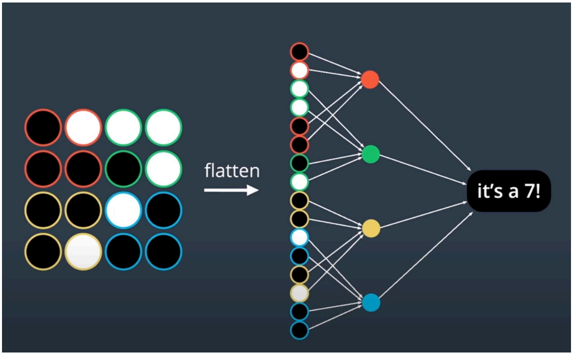


Figure 2: A simplified representation of a neural network prediction output.

Locally Connected Layers

Instead of fully connecting every node to every pixel, we can implement **sparse connectivity**. By dividing the input image into distinct local regions (e.g., four quadrants), we can assign specific hidden nodes to analyze only those small, local groups of pixels.

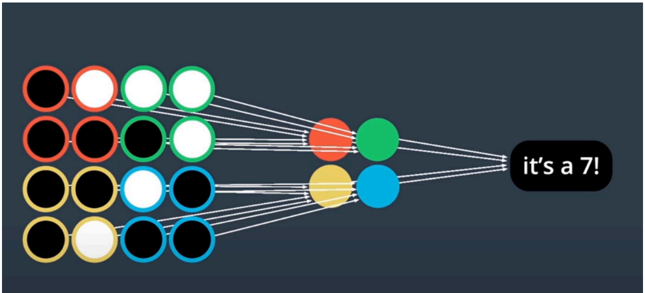


Figure 3: A locally connected layer where hidden nodes are responsible for specific color-coded regions (red, green, yellow, blue), significantly reducing the parameter count.

This approach offers two advantages:

- **Reduced Parameter Count:** Sparsely connected layers use far fewer parameters than densely connected ones.
- **Reduced Overfitting:** The model is forced to learn local patterns rather than memorizing the entire global input structure.

3. Parameter Efficiency: Weight Sharing

While expanding the number of hidden nodes helps detect more complex patterns, treating every region independently is inefficient. Visual patterns (patterns relevant to understanding the image) are generally translation invariant; a feature that appears in the top-left corner likely represents the same object if it appears in the bottom-right.

Weight Sharing leverages this invariance by enforcing that hidden nodes across different regions share the same weights.

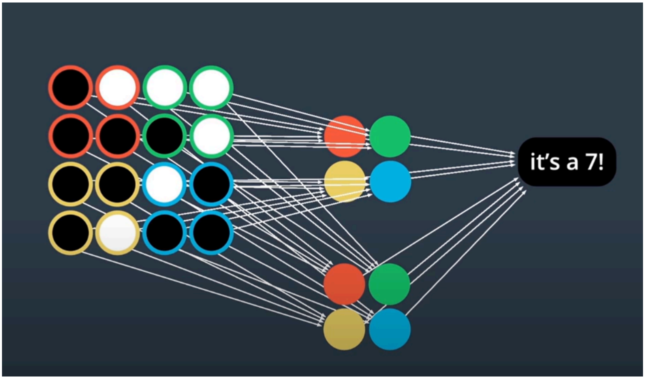


Figure 4: Concept of weight sharing. The network learns to recognize an object (e.g., a cat) regardless of its spatial position in the image, without needing to relearn the visual pattern for every possible location.

Conclusion

By integrating local connectivity and parameter sharing, we transition from densely connected MLPs to **Convolutional Layers**. This architecture efficiently handles high-dimensional image data, preserves spatial relationships, and robustly detects visual features regardless of their position in the image.

Technical Report: Fundamentals of Convolutional Neural Networks

1. Introduction: Preserving Spatial Integrity

Traditional neural networks process inputs individually, often neglecting the structural relationships between pixels. Convolutional Neural Networks (CNNs) represent a specialized class of neural networks designed to analyze images holistically or in patches, thereby retaining critical spatial information.

The core mechanism enabling this capability is the **Convolutional Layer**. Unlike fully connected layers, a convolutional layer applies a series of learnable filters (or kernels) to the input image.

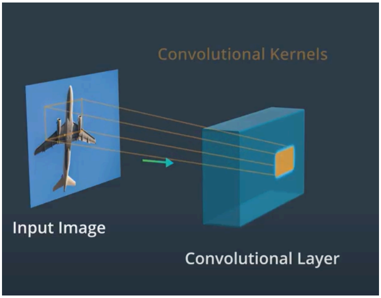


Figure 1: Illustration of a Convolutional Layer where kernels scan the input image to generate a feature map.

2. Feature Extraction via Convolutional Kernels

A convolutional layer functions by sliding different filters across the image. Each filter is designed to detect specific visual features. As these filters are applied, they transform the original input into a set of **filtered images** (feature maps), each highlighting different aspects of the original data.

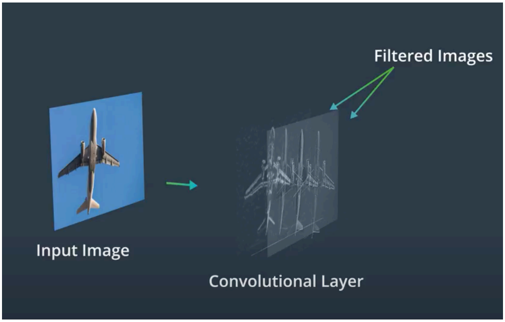


Figure 2: The process of applying multiple filters results in various distinct output maps, effectively decomposing the image into its constituent features.

3. Hierarchical Pattern Recognition

The primary objective of these filters is to extract meaningful features such as edges, colors, or geometric shapes. In the context of digit classification, for instance, the network learns to identify elementary spatial patterns—such as the specific curves and lines that distinguish the number "6" from other digits.

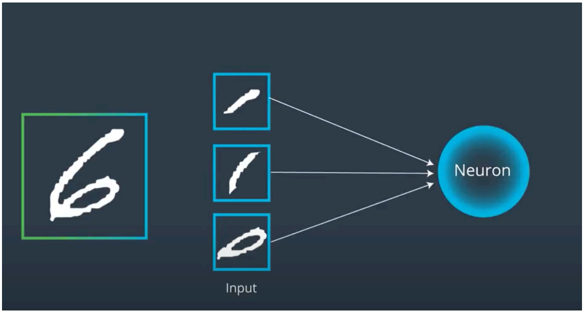


Figure 3: A CNN analyzing a handwritten digit by identifying distinct spatial components like curves and strokes.

Conclusion

By leveraging convolutional layers and filters, CNNs systematically transform raw pixel data into abstract feature representations. Initial layers focus on low-level features (edges, textures), while subsequent layers combine these spatial and color features to form complex representations capable of accurate classification. This foundational understanding of filters and transformations is essential for training and visualizing deep learning models.

Spatial Patterns and Intensity in Image Analysis

1. Understanding Spatial Patterns and Intensity

Spatial patterns in computer vision are primarily derived from color and shape. Shape itself is defined by patterns of **intensity**—the measure of light and dark (brightness) within an image. Understanding these intensity variations is crucial for detecting distinct objects.

2. Edge Detection via Contrast

Object identification relies on detecting boundaries where a subject ends and the background begins. These boundaries are characterized by **abrupt changes in intensity** (high contrast), such as a sharp transition from a dark region to a light one. These transitions define the edges of visual structures.

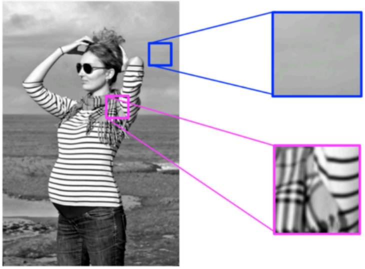
3. The Function of Image Filters

Convolutional Neural Networks (CNNs) automate this process using specialized **image filters**. These filters analyze groups of pixels to detect significant intensity gradients. By identifying these changes, the filters produce output maps that highlight edges and shapes, serving as the fundamental components for feature extraction in the network.

Technical Report: Image Frequency Analysis

1. Understanding Frequency as Rate of Change

In the domain of image processing, "frequency" is fundamentally defined as a **rate of change**. Rather than varying over time, images vary over space. Therefore, the frequency of an image describes how quickly distinct values—specifically brightness or intensity—change from one pixel to the next.



High and low frequency image patterns.

Figure 1: Comparison of high and low frequency patterns within a single image. The blue boundary highlights a smooth, uniform area (low frequency), while the magenta boundary highlights a textured, rapidly changing area (high frequency).

2. High vs. Low Frequency Components

High Frequency: A high-frequency image pattern is characterized by rapid transitions in intensity. This occurs where the level of brightness changes drastically between adjacent pixels. In the provided example, the checkered scarf and the striped shirt exhibit this behavior; the pixel values jump quickly from light to dark, signifying a high rate of change.

Low Frequency: Conversely, a low-frequency image pattern represents areas where brightness is relatively uniform or transitions very gradually. The background sky in the example illustrates this, as the intensity values remain consistent or shift slowly across the region.

Significance in Object Detection: Crucially, **high-frequency components correspond to the edges of objects**. Since edges are defined by sharp contrasts (sudden changes in intensity), detecting high-frequency patterns is a primary method for classifying and segregating distinct objects within an image.

Technical Report: High-Pass Filters and Kernel Convolution

1. High-Pass Filters and Edge Detection

In image processing, filters are employed to amplify specific features, such as object boundaries, while suppressing irrelevant information. A **high-pass filter** is specifically designed to sharpen images by enhancing high-frequency components—regions where pixel intensity changes rapidly (e.g., transitions from dark to light).

Application Example: Consider applying a high-pass filter to an image of a panda.

- **Uniform Areas:** In regions with little to no intensity change (solid dark or light furs), the filter outputs near-zero values, effectively turning these areas black.
- **Edges:** In areas where a pixel significantly differs in brightness from its neighbors, the filter leverages this contrast to effectively "draw" a line, highlighting the edge.

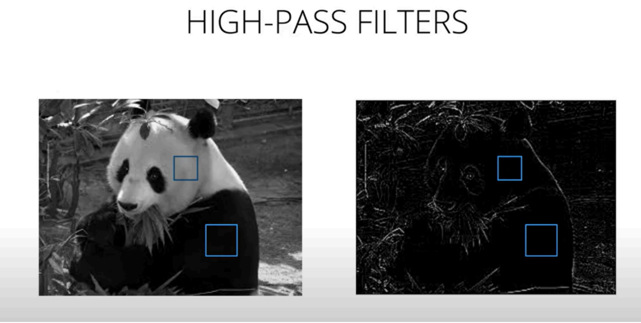


Figure 1: Demonstration of a high-pass filter. The original image (left) is processed to produce an edge-detected output (right), where only the boundaries of the panda and background foliage are visible.

2. The Mechanics of Kernel Convolution

The core operation behind these filters is **Kernel Convolution**. This involves sliding a small grid of numbers, known as a **convolution kernel**, over the input image.

Operational Steps:

1. **Centering:** The kernel (e.g., a 3x3 matrix) is centered over a specific pixel in the source image.
2. **Element-wise Multiplication:** Each value in the kernel is multiplied by the underlying pixel value.
3. **Summation:** The results are summed to produce a single new pixel value for the output image.

Edge Detection Kernel: A common high-pass kernel (shown below) typically has elements that sum to zero. This configuration ensures that the filter calculates the *difference* between neighboring pixels.

- **Positive Center:** Enhances the central pixel.
- **Negative Neighbors:** Subtracts the surrounding values. If the center and neighbors are similar, the sum is near zero (black). If they differ, the sum is large (bright edge).

CONVOLUTION

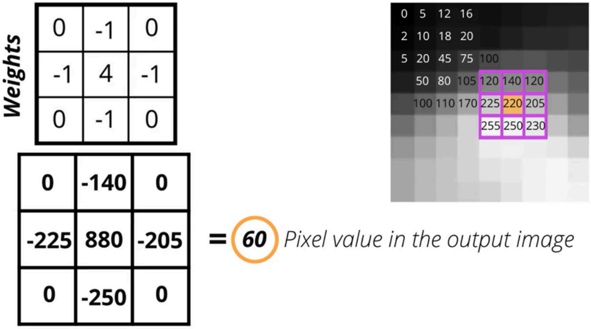


Figure 2: Structure of a 3x3 convolution kernel and the mathematical process of calculating a new pixel value (60) by applying weights to a local grid of pixels.

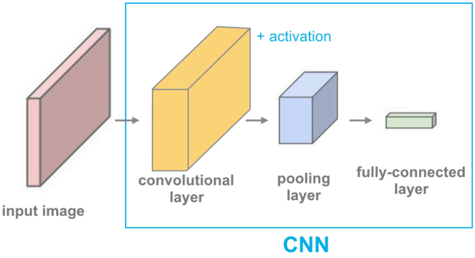
3. Edge Handling Strategies

Since a kernel cannot be centered on pixels at the extreme borders of an image (as it would overhang the image dimensions), specific strategies are required to handle these edge cases:

- **Extend:** The values of the nearest border pixels are replicated or "extended" outward to provide valid data for the convolution, preserving the image size.
- **Padding:** The image is surrounded by a border of constant values (commonly zeros/black pixels) to allow the kernel to process the edges.
- **Crop:** The convolution is only performed where the kernel fits entirely within the image. This results in an output image slightly smaller than the original.

The Importance of Filters

What you've just learned about different types of filters will be really important as you progress through this course, especially when you get to Convolutional Neural Networks (CNNs). CNNs are a kind of deep learning model that can learn to do things like image classification and object recognition. They keep track of spatial information and learn to extract features like the edges of objects in something called a convolutional layer. Below you'll see an simple CNN structure, made of multiple layers, below, including this "convolutional layer".

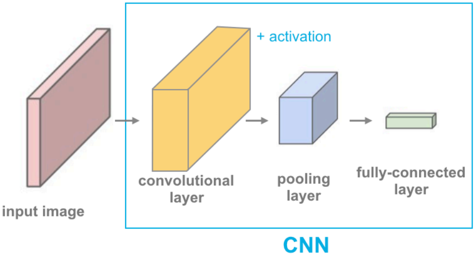


Convolutional Layer

The convolutional layer is produced by applying a series of many different image filters, also known as convolutional kernels, to an input image.

convolutional kernel (image filter)
input image

In the example shown, 4 different filters produce 4 differently filtered output images. When we stack these images, we form a complete convolutional layer with a depth of 4!



Learning

In the code you've been working with, you've been setting the values of filter weights explicitly, but neural networks will actually learn the best filter weights as they train on a set of image data. You'll learn all about this type of neural network later in this section, but know that high-pass and low-pass filters are what define the behavior of a network like this, and you know how to code those from scratch!

In practice, you'll also find that many neural networks learn to detect the edges of images because the edges of object contain valuable information about the shape of an object.

Convolutional Layers: Core Idea

- A single region of an image can contain multiple patterns (e.g., teeth, whiskers, tongue).
- To detect different patterns in the same region, **multiple filters** are used.
- Each filter is responsible for detecting a specific type of pattern.
- A convolutional layer typically contains **tens to hundreds of filters**.

Filters and Feature Maps

- Each filter produces its own output called a **feature map** (or activation map).
- Feature maps have the same structure as images: they are matrices of values.
- Feature maps are simplified representations of the original image, highlighting specific patterns.
- Brighter values in a feature map indicate stronger detection of the filter’s pattern.

Edge Detection Example

- Some filters learn to detect **vertical edges**, others **horizontal edges**.
- Edges usually appear as transitions between dark and light pixels.
- Edge-detecting filters are fundamental building blocks in CNNs.
- These filters help identify object boundaries and shapes.

Grayscale vs. Color Images

- Grayscale images are represented as **2D arrays** (height × width).
- Color (RGB) images are represented as **3D arrays** (height × width × depth).
- The depth corresponds to the three color channels: red, green, and blue.
- A color image can be seen as a stack of three grayscale images.

Convolution on Color Images

- Filters applied to color images are also **3D**, with one slice per color channel.
- The convolution operation is performed across all channels simultaneously.
- The result of convolving one filter with a color image is still **one feature map**.
- Using multiple filters results in multiple feature maps.

Stacking Feature Maps and Deep Representations

- Feature maps can be stacked to form a new 3D array.
- This stacked output can be fed into another convolutional layer.
- Deeper layers learn **patterns of patterns**, gradually building complex representations.
- Early layers detect simple features; deeper layers detect objects and high-level concepts.

Convolutional Layers vs. Dense Layers

- Dense layers are **fully connected**: every neuron connects to all neurons in the previous layer.
- Convolutional layers are **locally connected**: neurons connect only to small regions.
- Convolutional layers use **weight sharing**, reducing the number of parameters.
- Despite structural differences, both use weights, biases, and backpropagation.

Learning Filters Automatically

- Filters are initialized with **random values**.
- A loss function (e.g., categorical cross-entropy) defines the learning objective.
- During training, filters are updated via **backpropagation** to minimize the loss.
- CNNs automatically learn which patterns are important from the data.
- For example, a dataset with dogs leads to filters that respond to dog-like features.

Hyperparameters of Convolutional Layers

- The behavior of a convolutional layer is controlled by:
  - Number of filters
  - Filter size
  - Stride
  - Padding

Stride

- **Stride** is the number of pixels the filter moves each step.
- Stride = 1:
  - Filter moves one pixel at a time.
  - Output feature map has similar spatial size to the input.
- Stride = 2:
  - Filter skips pixels.
  - Output feature map is smaller (roughly half in width and height).

Edge Handling Problem

- With larger stride or filter size, filters may extend beyond image boundaries.
- When this happens, some output positions become undefined.
- This raises the question of how to handle image borders.

Padding Strategies

- **No padding (valid convolution)**:
  - Positions where the filter goes outside the image are discarded.
  - Output feature map is smaller.
  - Some edge information is lost.
- **Zero padding**:
  - The image is padded with zeros around the border.
  - The filter can cover all image regions.
  - Edge information is preserved.

Overall Summary

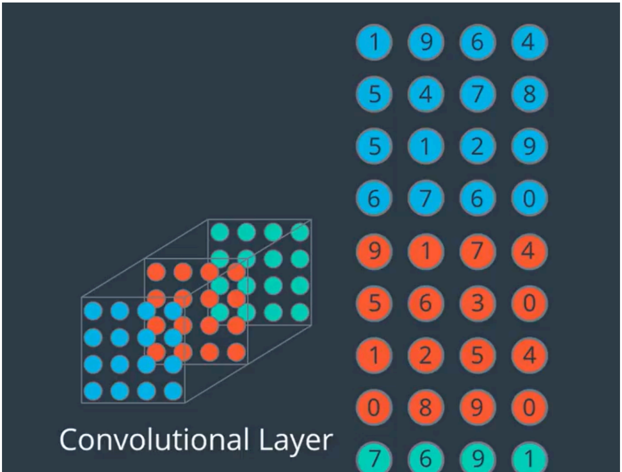
- Convolutional layers use multiple filters to detect different patterns.
- Feature maps highlight where patterns appear in the image.
- CNNs build hierarchical representations from simple to complex features.
- Filters are learned automatically from data.
- Stride and padding control the size and coverage of feature maps.
- Proper use of these concepts allows CNNs to effectively understand images.

Technical Report: Dimensionality Reduction via Pooling Layers

1. Handling Complexity in Convolutional Layers

A standard convolutional layer is composed of a stack of **feature maps**, with each map corresponding to a specific filter. As the complexity of a dataset increases, the network requires a larger number of filters to detect identifying patterns for various object categories.

This increase in filters results in a deeper stack of feature maps, significantly increasing the **dimensionality** of the layer. Higher dimensionality implies a greater number of parameters, which not only increases computational cost but also heightens the risk of **overfitting**, where the model memorizes training data rather than generalizing.



1	9	6	4
5	4	7	8
5	1	2	9
6	7	6	0
9	1	7	4
5	6	3	0
1	2	5	4
0	8	9	0
7	6	9	1

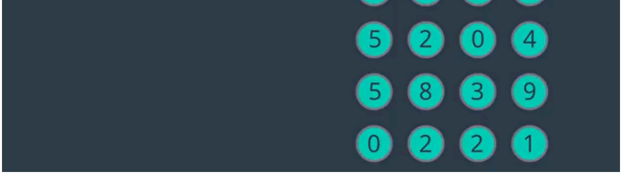


Figure 1: Visualization of a stack of feature maps in a convolutional layer. As the number of filters increases, the depth of this stack grows, necessitating dimensionality reduction.

2. Introduction to Pooling Layers

To mitigate high dimensionality, Convolutional Neural Networks (CNNs) employ **pooling layers**. These layers are designed to down-sample feature maps, reducing their spatial dimensions (width and height) while retaining the most critical information.

Max Pooling: The most common type is the **Max Pooling Layer**. It operates on each feature map independently using a sliding window approach, defined by a **window size** and a **stride**.

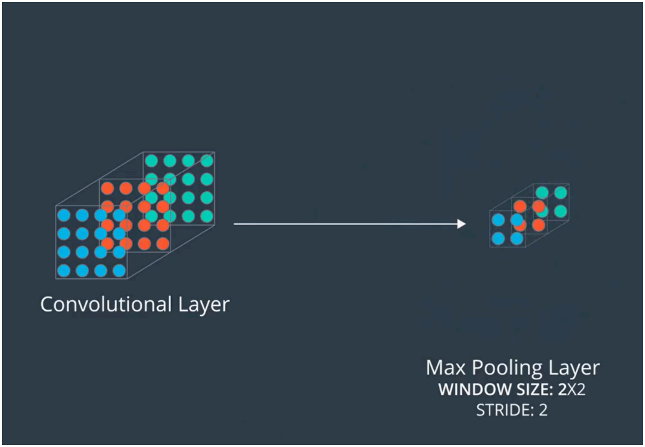
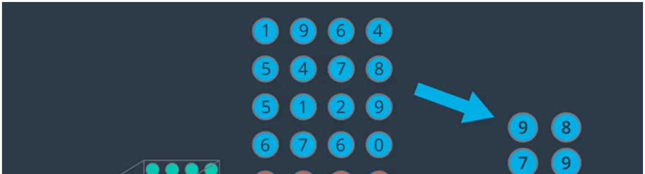


Figure 2: A 2x2 Max Pooling operation. The window slides over the input (blue), and for each position, the maximum value is extracted to form the output (green).

3. The Max Pooling Operation

The construction of a max pooling layer involves the following steps for each feature map:

1. **Window Placement:** A window (e.g., 2x2 pixels) is placed over a section of the image, starting from the top-left corner.
2. **Maximum Extraction:** The algorithm identifies the maximum pixel value within that window.
3. **Output Generation:** This maximum value becomes the corresponding pixel in the output layer.
4. **Stride:** The window moves by a set stride (e.g., 2 pixels) to the next position, and the process repeats.



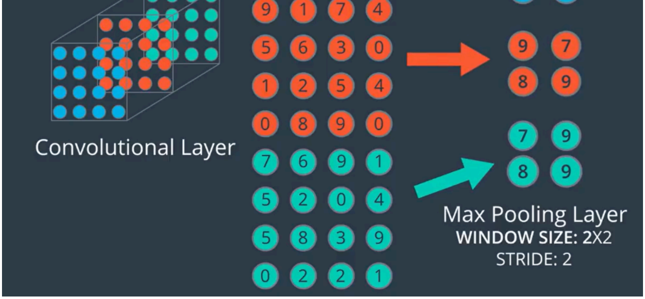


Figure 3: Detailed view of the max pooling calculation. In the highlighted 2x2 window containing values [1, 9, 5, 4], the maximum value (9) is selected.

Conclusion

The result of this process is a new stack of feature maps with the same depth (number of maps) as the input but with significantly reduced spatial dimensions. For instance, using a 2x2 window with a stride of 2 effectively halves both the width and height of the feature maps, providing a more compact and parameter-efficient representation of the image features.

Technical Report: Alternatives to Pooling and Capsule Networks

1. The Limitation of Pooling: Loss of Spatial Information

While pooling layers efficiently reduce dimensionality and prevent overfitting, they inherently discard pixel-level information. In tasks like image classification, this is generally acceptable. However, for more complex tasks like facial recognition, this loss of **spatial information**—the precise relationships between features—can be problematic.

The "Picasso Face" Problem: A standard CNN identifies a face by detecting key features (eyes, nose, mouth). Because standard pooling layers lose the precise spatial arrangement of these features, a CNN might incorrectly classify a distorted image (e.g., a face with a nose above the eyes) as a valid face simply because all the required parts are present.

2. Capsule Networks: Preserving Spatial Hierarchies

To address this, **Capsule Networks** were introduced. Unlike standard CNNs that may discard spatial context, Capsule Networks are designed to learn and preserve the spatial relationships between parts (e.g., the distance and angle between the nose and mouth).

Hierarchical Representation: Capsule networks construct a complete object representation through a hierarchy of parent and child nodes.

- **Leaf Nodes:** Detect individual parts (e.g., left eye, right eye, nose).
- **Parent Nodes:** Combine these parts to form larger structures (e.g., whole face) based on their relative positions.

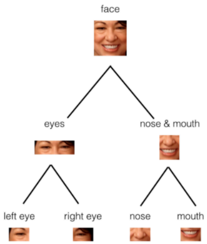


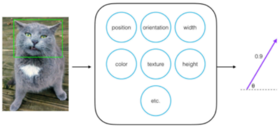
Figure 1: Hierarchical decomposition in a Capsule Network. Small parts (eyes, nose) detected by lower-level capsules are combined to verify the presence of the superior object (face).

3. Understanding Capsules and Vector Outputs

A "capsule" is a collection of neurons that provides a vector output, encoding two critical pieces of information about a detected feature:

1. **Magnitude (m):** Represents the **probability** that the part exists. The value ranges between 0 and 1.
2. **Orientation (θ):** Represents the **state** or properties of the part (e.g., pose, rotation, size, texture).

Vector Representation: This vector-based output allows the network to perform mathematical routing to determine if the detected parts properly align to form a coherent whole. Crucially, the **magnitude** (probability of existence) remains high even if the object is rotated or shifted, while the **orientation** (state properties) updates to reflect the change.



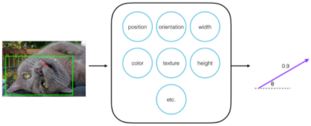


Figure 2: Capsule output as a vector. The length of the vector indicates the confidence of detection, while its direction encodes the object's spatial orientation and properties.

Technical Report: Designing a Complete CNN Architecture

1. Pre-processing: Standardization via Resizing

When dealing with real-world datasets, input images inevitably vary in spatial dimensions. However, like Multi-Layer Perceptrons (MLPs), Convolutional Neural Networks (CNNs) require a **fixed-size input**. Consequently, the first step in any pipeline is resizing all images to a unified square dimension (e.g., 32x32 pixels), ideally a number divisible by a large power of two.

Data Dimensions:

- **Color Images:** 3D array with height, width, and depth=3 (RGB).
- **Grayscale Images:** 2D spatial array, conceptually depth=1. Input arrays are typically much wider and taller than they are deep.

2. Architecture Goals: Deep and Narrow

The primary design philosophy of a CNN is to transform this input array (wide/tall, shallow) into an output that is **deep but spatially small**.

- **Convolutional Layers:** Increase the depth (number of feature maps) to extract complex hierarchical patterns.
- **Max Pooling Layers:** Decrease the spatial dimensions (x, y) to discard irrelevant background information and focus on content.

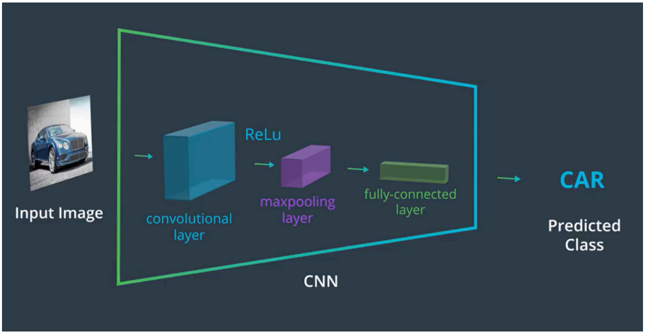


Figure 1: High-level view of a CNN architecture transforming an input image into a class prediction.

3. Layer Arrangement and Hierarchies

A complete architecture consists of sequential blocks of Convolutional and Pooling layers, followed by a Fully-Connected layer.

Convolutional Layers (Feature Extraction): Layers are stacked to discover hierarchies of patterns. Early layers detect simple patterns in the raw image, while deeper layers detect complex patterns from previous feature maps.

- **Hyperparameters:**
    - **Filters:** Number of output feature maps (Depth). Commonly doubles in sequence (e.g., 16 $\rightarrow$ 32 $\rightarrow$ 64).
    - **Filter Size:** Spatial extent of the kernel (e.g., 3x3).
    - **Stride:** Step size (usually 1).
    - **Padding:** Added border to maintain spatial dimensions (e.g., padding=1 for a 3x3 filter).

Max Pooling Layers (Dimensionality Reduction): Inserted periodically (e.g., after every 1-2 conv layers) to halve the spatial dimensions. A typical setting is a filter size of 2 and stride of 2.

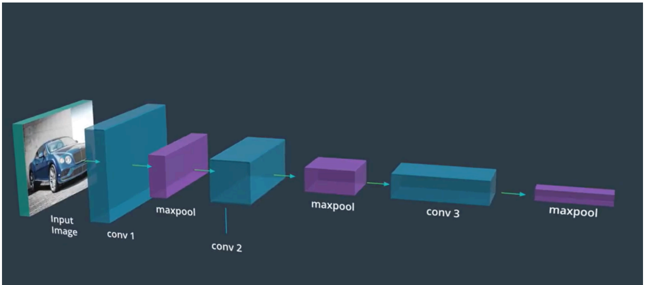


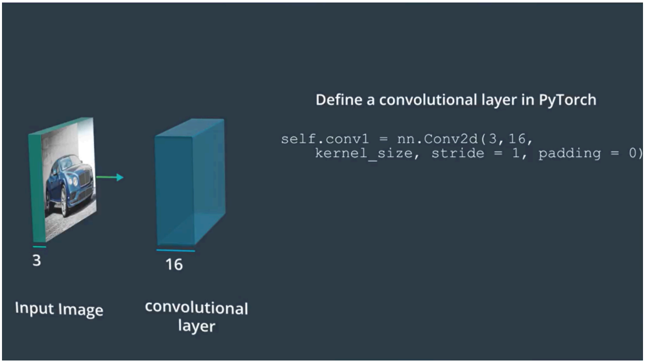
Figure 2: Volumetric visualization showing how the network increases depth (16 to 32 to 64) while reducing spatial width/height via max pooling.

4. Implementation Details

In frameworks like PyTorch, these layers are defined by specifying the input and output channels (depth). For example, the first layer accepts the image depth (e.g., 3 for RGB) and outputs the desired number of filters (e.g., 16).

PyTorch API (`nn.Conv2d`): To define a layer, we use the `nn.Conv2d` class with specific parameters: `nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)`

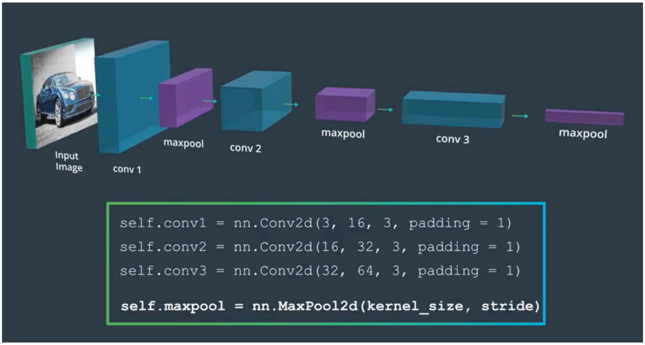
- `in_channels` : The depth of the input. For grayscale images, this is 1; for RGB, it is 3.
- `out_channels` : The desired depth of the output, representing the number of filtered images/feature maps to produce.
- `kernel_size` : The size of the square convolution kernel (commonly 3 for a 3x3 kernel).
- `stride` & `padding` : Parameters that determine the spatial size (x, y) of the output map. Defaults are typically 1 and 0, respectively.



Define a convolutional layer in PyTorch

```
self.conv1 = nn.Conv2d(3,16,
    kernel_size, stride = 1, padding = 0)
```

Figure 3: Defining a single convolutional layer in PyTorch, specifying input channels, output channels, and kernel hyperparameters.



```
self.conv1 = nn.Conv2d(3, 16, 3, padding = 1)
self.conv2 = nn.Conv2d(16, 32, 3, padding = 1)
self.conv3 = nn.Conv2d(32, 64, 3, padding = 1)

self.maxpool = nn.MaxPool2d(kernel_size, stride)
```

Figure 4: Complete architecture definition showing the sequence of Convolutional layers with increasing depth and Max Pooling layers for spatial reduction.

Convolutional Layers in PyTorch

To create a convolutional layer in PyTorch, you must first import the necessary module:

```
import torch.nn as nn
```

Then, there is a two part process to defining a convolutional layer and defining the feedforward behavior of a model (how an input moves through the layers of a network). First, you must define a Model class and fill in two functions.

init

You can define a convolutional layer in the **init** function of by using the following format:

```
self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)
```

forward

Then, you refer to that layer in the forward function! Here, I am passing in an input image x and applying a ReLU function to the output of this layer.

```
x = F.relu(self.conv1(x))
```

Arguments

You must pass the following arguments:

- in_channels - The number of inputs (in depth), 3 for an RGB image, for example.
- out_channels - The number of output channels, i.e. the number of filtered "images" a convolutional layer is made of or the number of unique, convolutional kernels that will be applied to an input.
- kernel_size - Number specifying both the height and width of the (square) convolutional kernel.

There are some additional, optional arguments that you might like to tune:

- stride - The stride of the convolution. If you don't specify anything, stride is set to 1.
- padding - The border of 0's around an input array. If you don't specify anything, padding is set to 0.

NOTE: It is possible to represent both kernel_size and stride as either a number or a tuple.

There are many other tunable arguments that you can set to change the behavior of your convolutional layers. To read more about these, we recommend perusing the official documentation(opens in a new tab).

Pooling Layers

Pooling layers take in a kernel_size and a stride. Typically the same value as is the down-sampling factor. For example, the following code will down-sample an input's x-y dimensions, by a factor of 2:

```
self.pool = nn.MaxPool2d(2,2)
```

---

forward

Here, we see that poling layer being applied in the forward function.

```
x = F.relu(self.conv1(x))
x = self.pool(x)
```

Convolutional Example #1

Say I'm constructing a CNN, and my input layer accepts grayscale images that are 200 by 200 pixels (corresponding to a 3D array with height 200, width 200, and depth 1). Then, say I'd like the next layer to be a convolutional layer with 16 filters, each filter having a width and height of 2. When performing the convolution, I'd like the filter to jump two pixels at a time. I also don't want the filter to extend outside of the image boundaries; in other words, I don't want to pad the image with zeros. Then, to construct this convolutional layer, I would use the following line of code:

```
self.conv1 = nn.Conv2d(1, 16, 2, stride=2)
```

Convolutional Example #2

Say I'd like the next layer in my CNN to be a convolutional layer that takes the layer constructed in Example 1 as input. Say I'd like my new layer to have 32 filters, each with a height and width of 3. When performing the convolution, I'd like the filter to jump 1 pixel at a time. I want this layer to have the same width and height as the input layer, and so I will pad accordingly. Then, to construct this convolutional layer, I would use the following line of code:

```
self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
```

Sequential Models

We can also create a CNN in PyTorch by using a Sequential wrapper in the **init** function. Sequential allows us to stack different types of layers, specifying activation functions in between!

```
def __init__(self):
        super(ModelName, self).__init__()
        self.features = nn.Sequential(
              nn.Conv2d(1, 16, 2, stride=2),
              nn.MaxPool2d(2, 2),
              nn.ReLU(True),

              nn.Conv2d(16, 32, 3, padding=1),
              nn.MaxPool2d(2, 2),
              nn.ReLU(True)
         )
```

Formula: Number of Parameters in a Convolutional Layer

The number of parameters in a convolutional layer depends on the supplied values of filters/out_channels, kernel_size, and input_shape. Let's define a few variables:

- K - the number of filters in the convolutional layer
- F - the height and width of the convolutional filters
- D_in - the depth of the previous layer

Notice that K = out_channels, and F = kernel_size. Likewise, D_in is the last value in the input_shape tuple, typically 1 or 3 (RGB and grayscale, respectively).

Since there are FFD_in weights per filter, and the convolutional layer is composed of K filters, the total number of weights in the convolutional layer is K*FFD_in. Since there is one bias term per filter, the convolutional layer has K biases. Thus, the __ number of parameters__ in the convolutional layer is given by K*FFD_in + K.

Formula: Shape of a Convolutional Layer

The shape of a convolutional layer depends on the supplied values of kernel_size, input_shape, padding, and stride. Let's define a few variables:

- K - the number of filters in the convolutional layer
- F - the height and width of the convolutional filters
- S - the stride of the convolution
- P - the padding
- W_in - the width/height (square) of the previous layer

Notice that K = out_channels, F = kernel_size, and S = stride. Likewise, W_in is the first and second value of the input_shape tuple.

The depth of the convolutional layer will always equal the number of filters K.

The spatial dimensions of a convolutional layer can be calculated as: (W_in−F+2P)/S+1

Flattening

Part of completing a CNN architecture, is to flatten the eventual output of a series of convolutional and pooling layers, so that all parameters can be seen (as a vector) by a linear classification layer. At this step, it is imperative that you know exactly how many parameters are output by a layer.

For the following quiz questions, consider an input image that is 130x130 (x, y) and 3 in depth (RGB). Say, this image goes through the following layers in order:

```
nn.Conv2d(3, 10, 3)
nn.MaxPool2d(4, 4)
nn.Conv2d(10, 20, 5, padding=2)
nn.MaxPool2d(2, 2)
```

Technical Report: Feature-Level Representation and Classification

1. The Goal: Content Over Style

The fundamental objective of a Convolutional Neural Network (CNN) is pattern discovery. Through a sequence of layers, a CNN transforms a raw input image array into a **feature-level representation** (or feature vector) that encodes the content of the image while discarding irrelevant details.

Abstraction Process: Consider two different images of cars. While they differ stylistically (color, angle, background), a classifier only cares that they contain the object "car".

- **Early Layers:** Capture style, texture, and pixel-level details.
- **Deep Layers:** Discard specific style/texture information in favor of general shapes and the presence of unique patterns (e.g., wheels, eyes, tails). As data propagates through the network, the representation becomes increasingly abstract, focusing solely on *what* is in the image rather than *how* it looks.

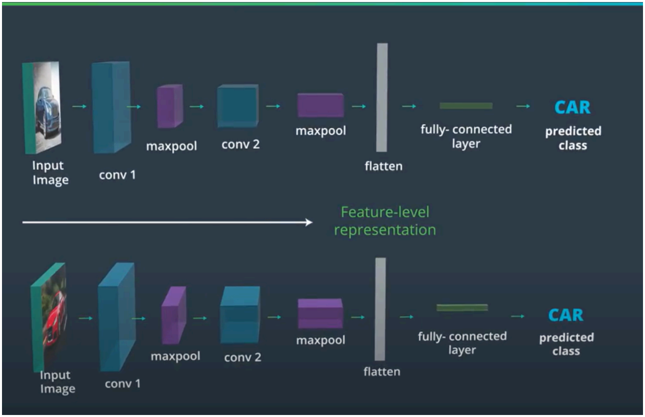


Figure 1: Evolution of data through a CNN. The input image is progressively transformed into a feature-level representation, which is then flattened and fed into fully connected layers for classification.

2. The Feature Vector

Once the image content has been distilled into a high-level representation (typically after the final pooling layer), the multidimensional array is **flattened** into a 1D feature vector. This vector summarizes the presence of critical components. For example:

- **Car:** The vector might strongly encode the presence of "wheels" and "metallic shapes".
- **Dog:** The vector encoding would reflect "eyes", "three legs", and "tail".

3. Classification via Fully Connected Layers

This feature vector is finally passed to one or more **fully connected layers**. The role of these layers is probabilistic deduction:

- If the feature vector indicates "wheels", the fully connected layer calculates a high probability for the class "Car".
- If the vector indicates "tail" and "fur", it deduces a high probability for "Dog".

Learned Understanding: Crucially, this "understanding" is not hard-coded. It is a result of **training** and **back-propagation**, where the model iteratively updates its weights (both in filters and fully connected layers) to minimize error and improve classification accuracy. The architecture simply provides the necessary structure for this learning to occur.

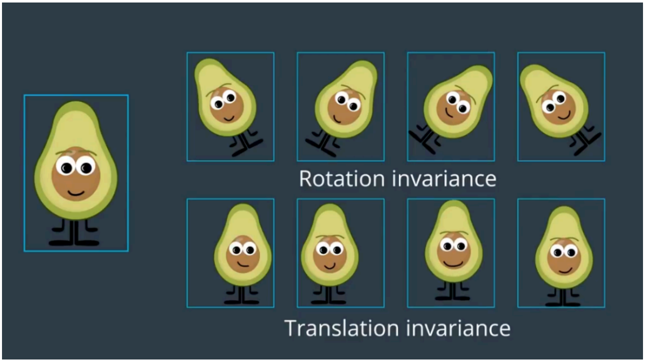
Image Augmentation

When we design an algorithm to classify objects in images, we have to deal with a lot of irrelevant information. We really only want our algorithm to determine if an object is present in the image or not. The size of the object doesn't matter, neither does the angle, or if I move it all the way to the right side of the image. It's still an image with an avocado.

In other words, we can say that we want our algorithm to **learn an invariant representation** of the image.

Types of Invariance

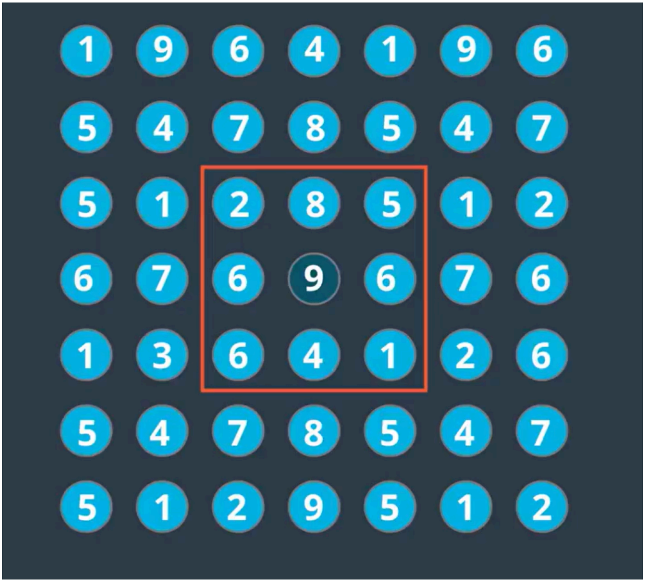
- **Scale Invariance**: We don't want our model to change its prediction based on the size of the object.
- **Rotation Invariance**: We don't want the angle of the object to matter.
- **Translation Invariance**: If I shift the image a little to the left or to the right, it's still an image with an avocado.



Max-Pooling and Translation Invariance

CNN's do have some built-in translation invariance. To see this, you'll need to first recall how we calculate max-pooling layers. Remember that at each window location, we took the maximum of the pixels contained in the window. This maximum value can occur anywhere within the window.

The value of the max-pooling node would be the same if we translated the image a little to the left, to the right, up, down, as long as the maximum value stays within the window.



1	9	6	4	1	9	6
5	4	7	8	5	4	7
5	1	2	8	5	1	2
6	7	6	9	6	7	6
1	3	6	4	1	2	6
5	4	7	8	5	4	7
5	1	2	9	5	1	2

The effect of applying many max-pooling layers in a sequence each one following a convolutional layer, is that we could translate the object quite far to the left, to the top of the image, to the bottom of the image, and still our network will be able to make sense of it all. This is truly a non-trivial problem. Recall that the computer only sees a matrix of pixels. Transforming an object's scale, rotation, or position in the image has a huge effect on the pixel values.

Data Augmentation

Thankfully, there's a technique that works well for making our algorithms more statistically invariant, but it will feel a little bit like cheating. The idea is this:

- If you want your CNN to be **rotation invariant**, you can just add some images to your training set created by doing **random rotations** on your training images.
- If you want more **translation invariance**, you can also just add new images created by doing **random translations** of your training images.

When we do this, we say that we have expanded the training set by **augmenting the data**.

Data augmentation will also help us to avoid overfitting. This is because the model is seeing many new images. Thus, it should be better at generalizing and we should get better performance on the test dataset.

Groundbreaking CNN Architectures

ImageNet

ImageNet is a database of over **10 million hand-labeled images** from 1,000 different categories. Since 2010, it has hosted the **ImageNet Large Scale Visual Recognition Competition (ILSVRC)**, where teams compete to build the best CNN for object recognition.

AlexNet (2012)

The first major breakthrough came from a team at the University of Toronto.

- **Training**: Used the best GPUs available at the time, taking about a week.
- **Key Innovations**: Pioneered the use of the **ReLU activation function** and **dropout** to avoid overfitting.

VGGNet (2014)

Developed by the Visual Geometry Group at Oxford University.

- **Versions**: VGG16 (16 layers) and VGG19 (19 layers).
- **Architecture**: A simple, elegant sequence of **3x3 convolutions** broken up by 2x2 pooling layers, ending with fully-connected layers.
- **Key Innovation**: Pioneered the exclusive use of small **3x3 convolution windows**, contrasting with AlexNet's larger 11x11 windows.

ResNet (2015)

Winner of the 2015 competition from Microsoft Research.

- **Depth**: incredibly deep, with the largest version having **152 layers**.
- **The Problem**: Previously, adding too many layers caused performance to decline due to the **vanishing gradient problem** (the gradient signal getting weakened as it passes through the network).
- **The Solution**: The team added **skip connections** (shortcuts) that allow the gradient signal to travel a shorter route.
- **Result**: Achieved **superhuman performance** in classifying images.

Visualizing CNNs

We've seen that CNNs achieve state-of-the-art and sometimes superhuman performance in object classification tasks. However, we don't always have a strong understanding of **how** these CNNs discover patterns in raw image pixels.

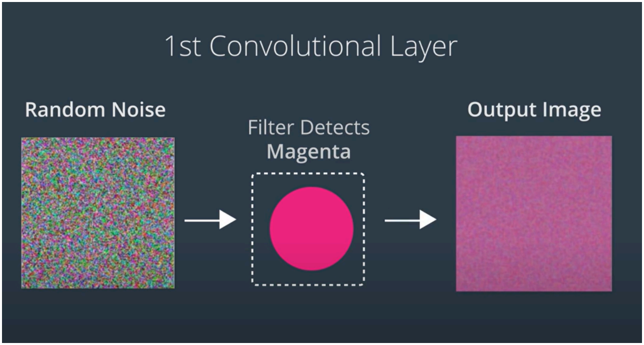
Visualizing Activation Maps

One technique for digging deeper is **visualizing the activation maps** and convolutional layers. This can help us understand why some architectures work while others don't.

Maximizing Activations (Feature Visualization)

Another technique involves taking the filters from our convolutional layers and constructing images that **maximize their activations**.

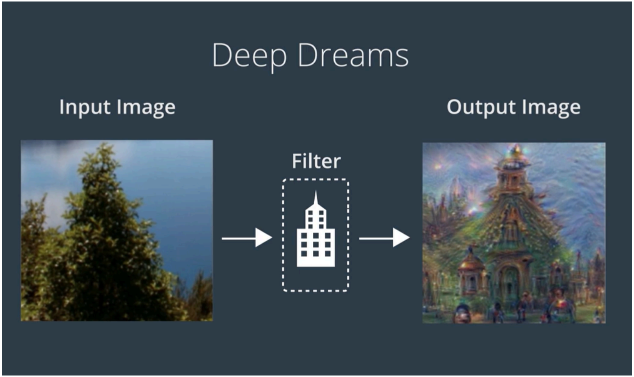
- You can start with an image containing **random noise** and gradually amend the pixels to make the filter more highly activated.
- **First Layers**: Typically detect general patterns like colors or edges.
- **Later Layers**: Activated by much more complicated patterns (circles, stripes, etc.).



Deep Dreams

Researchers at Google designed a technique called **Deep Dreams**.

- Instead of random noise, they start with a real picture (e.g., a tree).
- They choose a specific filter (e.g., one for detecting buildings).
- They amend the pixels to maximize that filter's activation.
- **Result**: An image that looks like a hybrid of the original object and the pattern the filter detects (e.g., a tree-building hybrid).



If we can understand more of what a CNN learns, we can help it to perform even better.

Summary of CNNs

So far, we've covered how to create neural networks for image classification and the specific steps a CNN follows.

Recap: How a CNN Works

1. **Input**: Takes in an input image.
2. **Feature Extraction**: Passes the image through several **convolutional and pooling layers**.
   - Result: A set of **feature maps** (reduced in size) that distill information about the content.
3. **Flattening**: These maps are flattened into a **feature vector**.
4. **Classification**: The vector is passed to **fully-connected linear layers**.
5. **Output**: Produces a probability distribution to predict the class label.

Key Takeaways

- **Performance**: CNNs achieve state-of-the-art results.
- **Versatility**: CNNs are **not restricted** to image classification. They can be applied to:
  - **Regression tasks**: E.g., detecting points on a face.
  - **Pose detection**: detecting human poses.
  - Any task with a fixed number of outputs.